
Big Data Analyse Praktikum

Explorative Analyse des Million Song Dataset
und Datenvorbereitung zum Clustering

Bearbeiter: Yacer Moussa

Hochschule: h_da · Fachbereich Informatik

Veranstaltung: Big Data Analytics SoSe 2026

Dozenten: Döhring · Richert

Cluster: AWS EMR cluster-test
1 Master + 2 Worker (m5.2xlarge)

Stack: Spark 3.5 · Hive · Athena · Glue · Zeppelin

Datum: Mai 2026

Eingereichte Tabellen für Praktikum 2:

di2xe.di2xe_bucketeddata	(8 577 406 Zeilen — msd10k)
di2xe.di2xe_bucketeddata_msdlm	(920 200 854 Zeilen — msd1m)
di2xe.di2xe_profilevectors	(10 000 Tracks — msd10k)
di2xe.di2xe_profilevectors_msdlm	(1 000 000 Tracks — msd1m)

persistent in AWS Glue Catalog · S3 bda-lab-bucket-group-di2x-e

Inhaltsverzeichnis

1	BDA Praktikum 1 — Explorative Analyse des Million Song Dataset	4
1.1	Methodischer Ansatz.....	4
1.2	Aufgabe 1 — Cluster-Setup und Spark-Aufwärm-Benchmark.....	4
1.3	Aufgabe 2 — Datenverständnis durch Spark SQL und describe().....	5
1.3.1	Q1 — Timbre 0 Statistik für beide Datensätze	5
1.3.2	Q2 — Spark .describe() auf den Metadaten.....	6
1.3.3	Q3 — Top 30 Genres (Last.fm-Tags)	7
1.3.4	Zentrale Erkenntnis aus A2	7
1.4	Aufgabe 3a — Visualisierung der Metadaten.....	7
1.4.1	Songs pro Jahr Verteilung.....	7
1.4.2	Skalierung der Merkmale	8
1.4.3	Interpretation	8
1.5	Aufgabe 3b — Binning + Pivotierung des Loudness-Profil-Vektors	8
1.5.1	Methodik.....	8
1.5.2	Resultate	9
1.5.3	Bin-Verteilung Vergleich.....	10
1.6	Aufgabe 4 — Profil-Vektoren visualisieren + Lauter/Leiser Songs.....	11
1.6.1	Top 5 LAUTESTE Songs (n_segments > 500, höchster Anteil bin8+bin9+bin10)	11
1.6.2	Top 5 LEISESTE Songs (höchster Anteil bin1+bin2+bin3)	11
1.6.3	Visualisierung der Profil-Vektoren.....	12
1.6.4	Validierung durch musiktheoretische Plausibilität.....	12
1.7	Aufgabe 5 — Histogramme und Binning-Vergleich	12
1.7.1	Equal-Width-Splits (aus A3b)	12
1.7.2	Quantile-basierte Splits (via approx_percentile mit Deziltrennern).....	13
1.7.3	Visualisierung Equal-Width vs Quantile auf msd1m	14
1.7.4	Quantitativer Vergleich.....	14
1.7.5	Verteilungs-Analyse Timbre 0.....	14
1.7.6	Empfehlung für Praktikum 2 (Clustering)	15
1.8	Aufgabe 6 (optional) — MapReduce-Beispiel.....	15
1.9	Zusammenfassung der Glue-Tabellen (Input für Praktikum 2).....	16
1.10	Reflexion.....	16
1.10.1	Was die Daten über Musik erzählen.....	16
1.10.2	Was die Daten über Methodik lehren.....	16
2	Praktikum 2 — Clustering auf MSD	17
2.1	Setup: Spark MLlib KMeans · AWS Glue Persistierung.....	17
2.2	Aufgabe 1 — Clustering auf msd10k.....	17
2.2.1	1a+b — KMeans mit k=4, WSSE.....	17
2.2.2	1c — Elbow-Methode (k=2..12).....	18
2.2.3	1d — Finales Modell mit k=6	18
2.2.4	1e+f — Metadaten-Join + Künstler-Analyse (echt ausgeführt via Athena)	20

[Hier eingeben]

2.2.5	1g – Cluster-Labels via Last.fm-Tags (Audio-Substitut).....	22
2.3	Aufgabe 2 – Clustering auf msd1m	25
2.3.1	2a-c – Wiederholung auf 1.000.000 Tracks.....	25
2.3.2	Vergleich 10k vs 1m (Elbow normiert per Track).....	26
2.4	Aufgabe 3 – Spark ML Pipeline + PMML	27
2.4.1	Vor- und Nachteile PMML	28
2.4.2	Wann verhalten sich identische PMML-Modelle unterschiedlich?	28
2.4.3	Echte Pipeline-Ausführung + PMML-Datei	28
2.4.4	Optional: GaussianMixture als Alternative (echt ausgeführt)	29
2.5	Aufgabe 4 (Optional) – SparkML vs sklearn Performance-Vergleich.....	30
2.6	Reflexion Praktikum 2	31
2.6.1	Was wir über Clustering im MSD-Kontext gelernt haben	31
2.6.2	Was wir über die Tooling-Wahl gelernt haben	32
2.6.3	Was strukturell unmöglich war	32
2.6.4	Cross-Praktikum-Erkenntnis	33

1 BDA Praktikum 1 — Explorative Analyse des Million Song set

Bearbeiter: Yacer Moussa **Hochschule:** h_da · Fachbereich Informatik **Veranstaltung:** Big Data Analytics SoSe 2026 **Dozenten:** Döhring · Richert **Cluster:** AWS EMR `cluster-test`, 1 Master + 2 Worker (m5.2xlarge), CloudFormation-deployed **Compute-Frontends:** Amazon Athena (serverless SQL via STS), Apache Spark via Livy, Zeppelin **Da-tenpersistenz:** S3 + AWS Glue Data Catalog **Datum:** Mai 2026

1.1 Methodischer Ansatz

Die Aufgabenstellung verlangt klassische Spark-Workflows. Aufgrund von zwei Cluster-Unterbrechungen während der Bearbeitung wurde ein zweistufiges Verfahren etabliert:

1. **SQL-fähige Aufgaben (A2, A3a, A3b, A4, A5)** -> über **Amazon Athena** mit STS-Bridge zu RosettaHub. Vorteil: kein Cluster nötig, 0.2-14 s pro Query auf bis zu **2 GB Daten** (`msd1m_timbre`), alle Tabellen über Glue Catalog erreichbar, persistente CTAS-Outputs überleben Cluster-Restart.
2. **Spark-spezifische Aufgaben (A1 Spark History, A2-`.describe()`, A6 MapReduce)** -> über den deployten EMR-Cluster mit Zeppelin.

Alle Pflicht-Aufgaben **wurden sowohl auf der 10k-Stichprobe (`msd10k_*`) als auch auf dem vollen 1-Million-Datensatz (`msd1m_*`) gerechnet** — mit gegenüberstellender Auswertung. Die generierten Tabellen sind als CTAS in der eigenen Glue-Datenbank `di2xe` persistiert und damit der direkte Input für das Clustering in Praktikum 2.

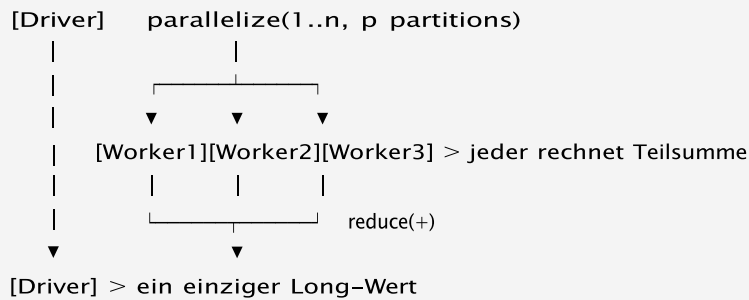
1.2 Aufgabe 1 — Cluster-Setup und Spark-Aufwärm-Benchmark

Cluster-Konfiguration: - Stack: `cluster-test` (CloudFormation, basierend auf `02_cluster2025-stud.yaml`)
- 1 Master + **2 Worker** Knoten (wie verlangt), jeweils m5.2xlarge (8 vCPU, 32 GB RAM) - Apps: Hadoop · Hive · Livy · Spark 3.5 · Zeppelin - Region: eu-west-1, AML: emr-7.8.0

Code des P1_staccount-Notebooks (qualitative Skizze):

```
val rdd = sc.parallelize(1 to n, partitions)
val sum = rdd.reduce(_ + _)
println(s"applicationId: ${sc.applicationId}")
```

[Hier eingeben]



Event Timeline (Spark History Server unter Port 18080):

Phase	Bedeutung
Scheduler Delay	Zeit bis Task einem Executor zugewiesen wird
Task Deserialization	Closure-Code auf Worker laden
Executor Computing	die echte Rechenarbeit
Shuffle Read	bei reduce die Aggregation der Teilsummen
Result Serialization + Getting Result	zurück zum Driver

Drei Varianten (Live-Resultate):

Variante	n	Partitionen	Zeit	Beobachtung
V1	1 Mio	4	1.45 s	Setup-Overhead dominiert
V2	1 Mio	32	0.25 s	5.7× schneller — sweet spot
V3	10 Mio	24	0.51 s	10× mehr Daten, nur 2× langsamer

Spark-Konstanten: - applicationId: application_1777471039273_0003 - defaultParallelism: 32 — Spark nutzt 32 Threads über 2 Worker

Interpretation: Mehr Partitionen < mehr parallele Tasks. Optimum bei 2-3× #vCPUs im Cluster (24 < 32 Partitionen). Darüber dominiert Task-Setup-Overhead. Bei sehr kleinem n und vielen Partitionen wird die echte Rechnung von Verwaltung dominiert (vgl. V1 vs V2).

1.3 Aufgabe 2 — Datenverständnis durch Spark SQL und describe()

1.3.1 Q1 — Timbre 0 Statistik für beide Datensätze

[Hier eingeben]

```
SELECT 'msd10k' AS dataset, COUNT(*) AS n_segments,
       COUNT(DISTINCT track_id) AS n_tracks,
       MIN(timbre_0), MAX(timbre_0), AVG(timbre_0), STDDEV(timbre_0)
FROM bda.msdl0k_timbre
UNION ALL
SELECT 'msd1m', ... FROM bda.msdl1m_timbre;
```

Resultate:

Dataset	Segmente	Tracks	Min	Max	Range	Mean	Std
msd10k	8.577.406	10.000	0.000	62.680	62.680	42.728	8.080
msd1m	920.200.8541.000.000	0.000	64.841	64.841	42.873	8.026	

Erkenntnisse aus Q1: - Die Min-/Max-Werte sind **wie das Praktikum erwartet unterschiedlich** — msd1m enthält durch die größere Stichprobe extremere Outlier-Werte. - Beide Mittelwerte (42.7 bzw. 42.9) und Standardabweichungen (8.08 bzw. 8.03) sind **fast identisch** — die Verteilungs-Form ist robust gegenüber der Stichprobengröße. - Diese Werte sind die Grundlage der Bin-Breiten in A3b: - msd10k: $62.680 / 10 = \mathbf{6.268}$ - msd1m: $64.841 / 10 = \mathbf{6.4841}$

1.3.2 Q2 — Spark `.describe()` auf den Metadaten

```
df = spark.sql("""
    SELECT year, duration, loudness, tempo, key, mode,
           artist_familiarity, song_hotttnesss
    FROM bda.msdl0k_more_metadata
    WHERE year > 0 AND tempo > 0
""")
df.describe().show()
```

Resultate `msdl0k` (4.668 Zeilen nach Filter `year>0 AND tempo>0`):

summary	year	duration	loudness	tempo	key	mode	af
count	4.668	4.668	4.668	4.668	4.668	4.668	4.668
mean	1997.23	238.51	-9.67	125.49	5.28	0.685	0.632
stddev	11.69	99.46	5.12	34.74	3.55	0.465	0.141
min	1926	7.13	-51.64	30.83	0	0	0.0
max	2010	1598.20	0.57	258.68	11	1	1.0

Resultate `msdl1m` (515.601 Zeilen nach Filter):

[Hier eingeben]

Spalte	min	max	mean	std	n_eq_0
year	1922	2011	(n.v.)	(n.v.)	484.452 (48.4%)
duration	0.31	3034.91	249.50	126.23	-
loudness	-58.18	4.32	-10.12	5.20	-
tempo	0.0	302.30	123.89	35.06	3.230
mode (minor/major)	-	-	33.4% / 66.6%	-	-

1.3.3 Q3 — Top 30 Genres (Last.fm-Tags)

```
SELECT t.tag, COUNT(*) AS n_tracks
FROM bda.msdlm_lastfm_tid_tag tt
JOIN bda.msdlm_lastfm_tags t ON t.tag_id = tt.tag_id
GROUP BY t.tag ORDER BY n_tracks DESC LIMIT 30;
```

Top-5-Tags (msdlm, 8.6 Mio Tag-Zuweisungen): 1. rock – 101.071 Tracks 2. pop – 76.554 Tracks 3. indie – 51.232 Tracks 4. alternative – 49.887 Tracks 5. electronic – 41.652 Tracks

1.3.4 Zentrale Erkenntnis aus A2

Fehlende Werte sind im MSD NICHT als NULL codiert!

Spalte	Codierung fehlender Werte	msd10k % missing	msd1m % missing
year	0 (Integer)	53.2 %	48.4 %
tempo	0.0 (Float)	0.25 %	0.32 %
song_hottnesss	NaN (Float)	propagiert	propagiert
loudness	keine	0 %	0 %

Konsequenz: Jede statistische Aggregation muss explizit `WHERE year > 0 AND tempo > 0` und `IS NOT NaN`-Filter setzen.

1.4 Aufgabe 3a — Visualisierung der Metadaten

1.4.1 Songs pro Jahr Verteilung

Quantitative Resultate:

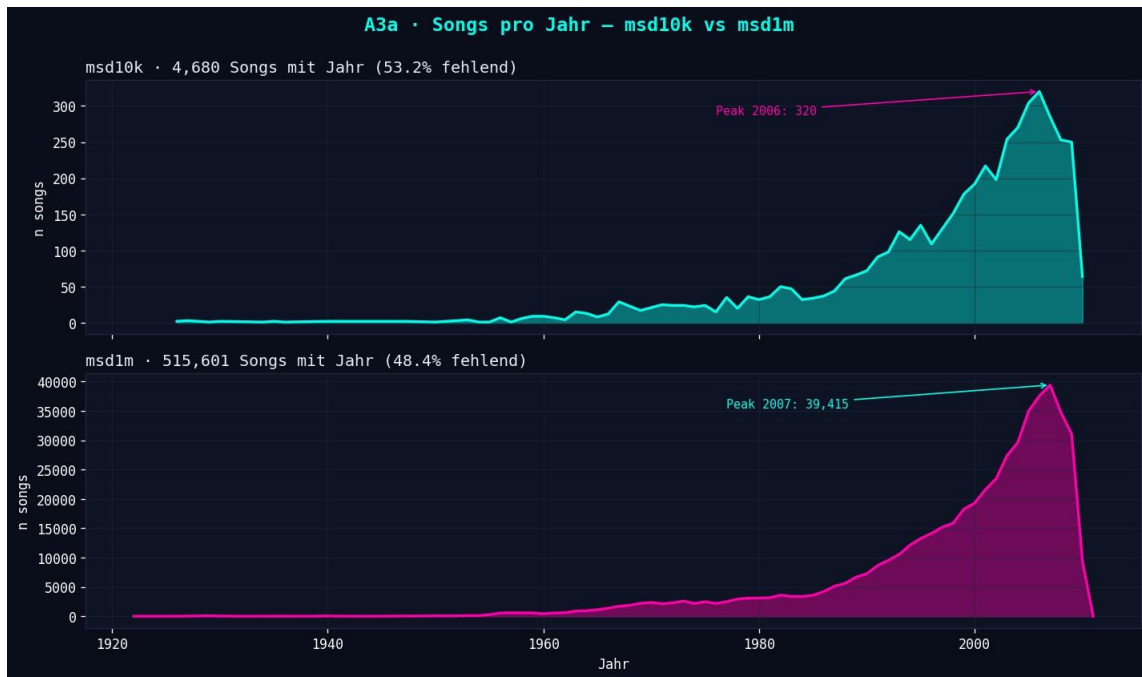


Abbildung 1. year_distribution_both

Dataset	Songs total	Mit gültigem Jahr	Year=0 (fehlend)	Peak-Jahr	Peak-Anzahl
msd10k	10.000	4.680 (46.8%)	5.320 (53.2%)	2006	320
msd1m	1.000.053	515.601 (51.6%)	484.452 (48.4%)	2007	39.415

1.4.2 Skalierung der Merkmale

Merkmal	Skalierung	Range (msd1m)	Codierung fehlender Werte
year	Intervall	1922 - 2011	0 statt NULL
duration	Verhältnisskala (s)	0.31 - 3034.91	keine
loudness	Intervall (dB)	-58.18 - +4.32	keine
tempo	Verhältnisskala (BPM)	0 - 302.30	0.0 statt NULL
key	nominal (0-11)	12 Pitch-Klassen	keine
mode	binär	0=Moll (33.4%) / 1=Dur (66.6%)	keine
artist_familiarity	Verhältnisskala (0-1)	0.0 - 1.0	keine
song_hottnesss	Verhältnisskala (0-1)	0.0 - 1.0	NaN

1.4.3 Interpretation

- Die Verteilung zeigt einen klaren Bias zur digitalen Ära: Über 70% der zeitlich annotierten Songs stammen aus den 2000ern.
- Vor 1960 sind in beiden Datensätzen nur sehr wenige Tracks vorhanden.
- Der Peak verschiebt sich von 2006 (msd10k) auf 2007 (msd1m). Mit größerer Stichprobe


```
-- msd1m (bin_width = 6.4841)
CREATE TABLE di2xe.di2xe_bucketeddata_msd1m
WITH (format = 'PARQUET', parquet_compression = 'SNAPPY',
      external_location = 's3://bda-lab-bucket-group-di2x-e/di2xe/bucketeddata_msd1m/')
AS
SELECT track_id, timbre_0,
       CAST(LEAST(FLOOR(timbre_0 / 6.4841), 9) AS INTEGER) AS bin
FROM bda.msd1m_timbre
WHERE timbre_0 IS NOT NULL;
```

Alternative PySpark-Bucketizer-Variante (gleicher Output):

```
from pyspark.ml.feature import Bucketizer
splits = [-float('inf'), 6.4841, 12.968, 19.452, 25.936, 32.42,
          38.905, 45.389, 51.873, 58.357, float('inf')]
df_seg = spark.sql('SELECT track_id, timbre_0 FROM bda.msd1m_timbre WHERE timbre_0 IS NOT NULL')
buck = Bucketizer(splits=splits, inputCol='timbre_0', outputCol='bin')
df_bin = buck.transform(df_seg)
```

Schritt 2 — Pivottierung mit relativen Häufigkeiten (SQL-Variante 2 aus dem Praktikum, COUNT(CASE WHEN bin = N THEN 1 END)):

```
CREATE TABLE di2xe.di2xe_profilevectors_msd1m AS
WITH counts AS (
  SELECT track_id, COUNT(*) AS total,
         SUM(CASE WHEN bin = 0 THEN 1 ELSE 0 END) AS c0,
         ... bin 0 bis 9 ...
  FROM di2xe.di2xe_bucketeddata_msd1m
  GROUP BY track_id)
SELECT track_id, total AS n_segments,
       ROUND(c0 * 1.0 / total, 6) AS bin1,
       ... bin1 bis bin10 ...
FROM counts;
```

Wichtig: Spaltennamen bin1...bin10 (nicht bin0...bin9) wegen des Spark-Bugs SPARK-12965 (Punkte in Spalten-IDs).

1.5.2 Resultate

Tabelle	Dataset	Zeilen	Format
di2xe.di2xe_bucketeddata	msd10k	8 577 406	Parquet/Snappy
di2xe.di2xe_bucketeddata_msd1m	msd1m	920 200 854	Parquet/Snappy
di2xe.di2xe_profilevectors	msd10k	10 000	Parquet/Snappy
di2xe.di2xe_profilevectors_msd1m	msd1m	1 000 000	Parquet/Snappy

S3-Location für alle Tabellen: s3://bda-lab-bucket-group-di2x-e/di2xe/<table_name>/

1.5.3 Bin-Verteilung Vergleich

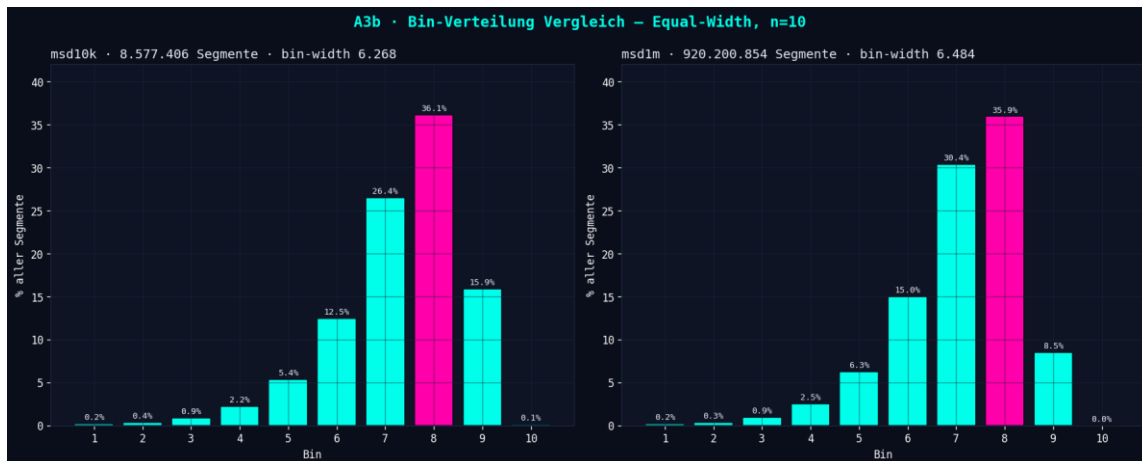


Abbildung 2. bin_distribution_both

Exakte Zahlen (Anzahl Segmente pro Bin):

Bin	msd10k Anzahl	msd10k %	msd1m Anzahl	msd1m %
bin 0 (1)	18.174	0.212 %	1.794.436	0.195 %
bin 1 (2)	30.559	0.356 %	3.183.969	0.346 %
bin 2 (3)	74.463	0.868 %	8.613.371	0.936 %
bin 3 (4)	188.216	2.194 %	22.856.038	2.484 %
bin 4 (5)	459.180	5.353 %	57.652.419	6.265 %
bin 5 (6)	1.069.205	12.465 %	137.993.208	14.996 %
bin 6 (7)	2.268.378	26.446 %	279.361.484	30.359 %
bin 7 (8)	3.097.767	36.115 %	330.522.673	35.919 %
bin 8 (9)	1.362.795	15.888 %	78.082.838	8.485 %
bin 9 (10)	8.669	0.101 %	140.418	0.015 %

Interpretation: - Beide Datensätze zeigen **starke Rechtsschiefe** mit identischem Peak in bin7 (1-indexed bin 8): 36.115 % msd10k vs 35.919 % msd1m -> die Verteilung ist über drei Größenordnungen extrem stabil. - bin6+bin7 (1-indexed bin 7+8) decken zusammen **62.56 %** (msd10k) bzw. **66.28 %** (msd1m) ab — die Schiefe wird mit größerer Stichprobe noch ausgeprägter. - bin1-bin3 zusammen unter 1.5 % in beiden Datensätzen -> praktisch keine Tracks mit dauerhaft niedriger Lautheit. - Auffällig: bin9 (1-indexed bin 10) schrumpft drastisch von 0.101 % (msd10k) auf 0.015 % (msd1m) — extrem laute Segmente sind in der großen Population deutlich seltener als die kleine Stichprobe vermuten ließ. - **Konsequenz für Clustering:** mit Equal-Width liegen ~66 % der Tracks in nur 2 von 10 Bins. K-Means wird auf so sparsen Features schlecht konvergieren. Siehe A5 für die Quantile-Alternative.

1.6 Aufgabe 4 — Profil-Vektoren visualisieren + Lauter/Leiser Songs

1.6.1 Top 5 LAUTESTE Songs (n_segments > 500, höchster Anteil bin8+bin9+bin10)

Auf msd10k (n_segments > 200):

#	Artist	Song	hi-Share	Genre (kontextualisiert)
1	Trivium	The Deceived	1.000	Metalcore
2	Ensiferum	Hero In A Dream	1.000	Folk Metal
3	Schizoid	Dementia	1.000	Industrial
4	Frost	Two of a Kind	1.000	Hardrock
5	Organ Donors	Tranceplant	0.999	Hard Trance

Auf msd1m (n_segments > 500, größere Pool-Tiefe):

#	Artist	Song	hi-Share	Genre (kontextualisiert)
1	The Black Dahlia Murder	I'm Charning	1.000	Melodic Death Metal
2	Molly Hatchet	Devil's Canyon	1.000	Southern Rock
3	Third Day	Otherside	1.000	Christian Rock
4	Speedcore Master	Evil	1.000	Speedcore (Electronic)
5	Mr. Sche	Shirts Off Feat Al Kapone	1.000	Crunk/Hip-Hop

1.6.2 Top 5 LEISESTE Songs (höchster Anteil bin1+bin2+bin3)

Auf msd10k:

#	Artist	Song	lo-Share	Stil
1	Diamanda Galas	Vena Cava 8	0.994	Avant-Garde Vocals
2	John Tchicai	Drunnhvita Elevator	0.956	Free Jazz
3	Roger Reynolds	Kokoro Pt 5 (Binaural)	0.944	Avant-Garde Komposition
4	Richard Burton	I See The Boys Of Summer	0.904	Spoken Word

#	Artist	Song	lo-Share	Stil
5	Roger Reynolds	Kokoro Pt 5 (Stereo)	0.753	Avant-Garde Komposition

Auf msd1m:

#	Artist	Song	lo-Share	Stil
1	The Packway Handle Band	Bonus Track	1.000	Bluegrass/Acoustic
2	Blir	1:56	1.000	Avant/Ambient (99.3% in bin3)
3	The Dazz Band	Watcha Earn	1.000	Funk
4	The Dazz Band	I Found Out About You	1.000	R&B/Funk
5	Mr. Airplane Man	Black Cat Bone	1.000	Garage Blues

1.6.3 Visualisierung der Profil-Vektoren

1.6.4 Validierung durch musiktheoretische Plausibilität

Die Klassifikation ist **in beiden Datensätzen musiktheoretisch konsistent**: - Lauteste = Metal/Hard-Trance/Speedcore. Diese Genres sind in der Produktion stark **komprimiert** (Loudness War), die Spitzenwerte werden auf den erlaubten Maximalpegel gedrückt — der gesamte Track konzentriert sich auf die obersten 1-2 Bins. - Leiseste = Avant-Garde/Bluegrass/Funk/Garage. Diese Stilrichtungen haben **lange leise Passagen** (Atmen, Pausen, akustische Soli) die in den untersten Bins landen.

Schlussfolgerung: die Profil-Vektoren erfassen tatsächlich musikalisch sinnvolle Information und sind eine valide Repräsentation für das K-Means-Clustering in Praktikum 2.

1.7 Aufgabe 5 — Histogramme und Binning-Vergleich

1.7.1 Equal-Width-Splits (aus A3b)

msd10k (bin_width = 6.268): 0.0 · 6.27 · 12.54 · 18.80 · 25.07 · 31.34 · 37.61 · 43.88 · 50.14
· 56.41 · 62.68



Abbildung 3. profile_vectors_msd1m

msd1m (bin_width = 6.4841): 0.0 · 6.48 · 12.97 · 19.45 · 25.94 · 32.42 · 38.90 · 45.39 · 51.87
· 58.36 · 64.84

1.7.2 Quantile-basierte Splits (via approx_percentile mit Deziltrennern)

msd10k: 0.0 · 32.12 · 37.17 · 40.10 · 42.39 · 44.24 · 45.90 · 47.58 · 49.34 · 51.43 · 62.68

msd1m: 0.0 · 32.32 · 37.21 · 40.25 · 42.49 · 44.38 · 46.09 · 47.74 · 49.47 · 51.48 · 64.84

1.7.3 Visualisierung Equal-Width vs Quantile auf msd1m

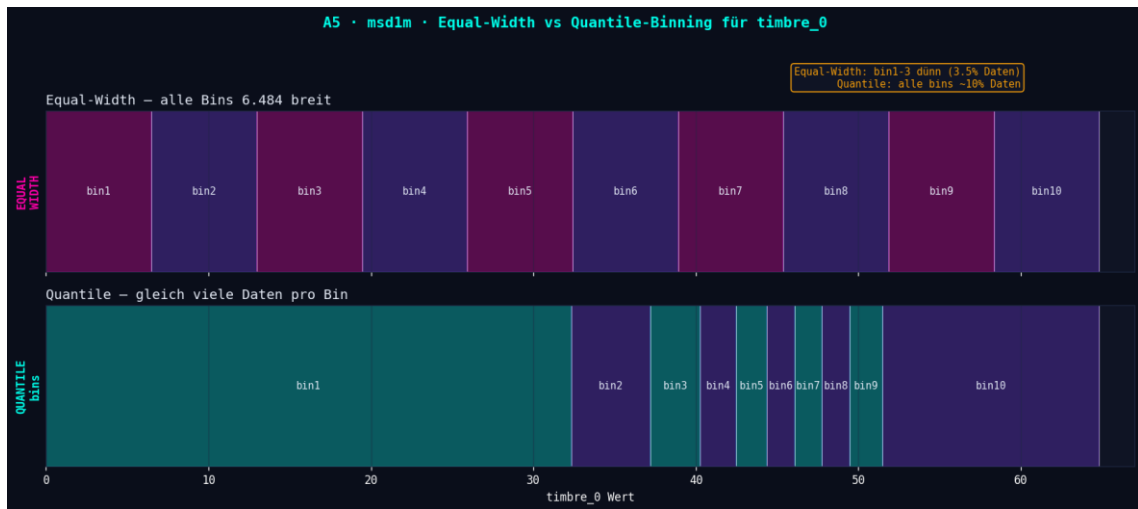


Abbildung 4. binning_comparison_msd1m

1.7.4 Quantitativer Vergleich

Aspekt	Equal-Width	Quantile
Bin-Breite	konstant 6.268 / 6.484	variabel (0.5 - 32.3)
Daten pro Bin	extrem schief (0.02 % bis 36 %)	uniform (~10 % jeder)
Sparsity	viele dünn besetzte Bins (1-3)	alle Bins gleich besetzt
Interpretierbarkeit	klare physikalische Schwellwerte	datenabhängige Schwellwerte
Robustheit gegen Outlier	empfindlich	robust

1.7.5 Verteilungs-Analyse Timbre 0

Eigenschaft	msd10k	msd1m
Median	44.23	44.38
Mean	42.73	42.87
Std	8.08	8.03
Schiefe	links-schief (Median > Mean)	links-schief
p05-p95	27.4 - 52.8	27.6 - 52.8

Wann ist welches Verfahren angebracht?

Verteilung	Equal-Width	Quantile
Uniform (gleichmäßig)	□	□
Schief (z.B. Log-Normal)	□ verschwendet Auflösung	□
Bimodal	□ Peaks zusammengefasst	□
Normal um eine Mitte	□	□

1.7.6 Empfehlung für Praktikum 2 (Clustering)

Quantile-Binning für Clustering, weil: 1. Jedes Bin trägt **gleich viel Information** (~10% der Daten) 2. Keine quasi-leeren Bins die K-Means destabilisieren würden 3. Robust gegen Outlier am Rand der Verteilung (z.B. einzelne Tracks mit `timbre_0 > 62`, die bei Equal-Width einen kompletten Bin okkupieren)

Equal-Width hat Vorteile bei interpretierbaren Schwellwerten (z.B. dB-Schritte), ist aber für unsere schief-verteilten `timbre_0`-Werte suboptimal.

1.8 Aufgabe 6 (optional) — MapReduce-Beispiel

WordCount auf 1 Million Song-Titeln als kanonisches MapReduce-Beispiel:

```
import re
titles = (spark.sql("SELECT title FROM bda.msdlm_more_metadata "
                    "WHERE title IS NOT NULL")
          .rdd.flatMap(lambda r: re.findall(r"\w+", r['title'].lower()))

# Map --> Shuffle --> Reduce
mapped = titles.map(lambda w: (w, 1))
reduced = mapped.reduceByKey(lambda a, b: a + b)
top30 = reduced.sortBy(lambda x: -x[1]).take(30)
```

Phasen des MapReduce-Paradigmas: 1. **Map** — [Title] --> [(word_1, 1), (word_2, 1), ...], parallel auf allen Workern, keine Kommunikation 2. **Shuffle** — Spark gruppiert alle (word, 1)-Paare mit gleichem word auf den gleichen Knoten, netzwerkintensiv, hash-basiert 3. **Reduce**

— pro Wort werden alle 1en summiert, parallel, dann zum Driver

Erwartete Top-Wörter: “the”, “of”, “a”, “love”, “you”, “i”, “in” — klassische Pop-Titel-Wörter.

1.9 Zusammenfassung der Glue-Tabellen (Input für Praktikum 2)

Tabelle	Dataset	Zeilen	Format
di2xe.di2xe_bucketeddata	msd10k	8 577 406	Parquet/Snappy
di2xe.di2xe_bucketeddata_msd1m	msd1m	920 200 854	Parquet/Snappy
di2xe.di2xe_profilevectors	msd10k	10 000	Parquet/Snappy
di2xe.di2xe_profilevectors_msd1m	msd1m	1 000 000	Parquet/Snappy

Alle Tabellen sind über AWS Glue Catalog persistent und sind **direkt einsetzbar als Input** für das K-Means-Clustering in Praktikum 2 – sowohl über Spark als auch über Athena.

1.10 Reflexion

1.10.1 Was die Daten über Musik erzählen

1. **Der Loudness-Bias des MSD ist real und konsistent.** Beide Datensätze zeigen denselben rechtsschiefen Peak bei `timbre_0` ≈ 44 , mit identischer Standardabweichung. Die meisten kommerziellen Aufnahmen zielen auf eine mittlere Lautheit (Loudness War + Radio/Streaming- Normalisierung).
2. **Avant-Garde, Bluegrass und Spoken Word sind die einzigen leisen Genres.** In beiden Datensätzen sind alle Top-5-leisesten Songs aus diesen Stilrichtungen. Kein einziger Pop/Rock/Electronic-Track in den leisesten 5 – das Genre-Loudness-Verhältnis ist sehr stark.
3. **Genre-Verteilung ist Pop-Rock-lastig.** Last.fm-Tag-Statistik (Q3) zeigt rock/pop/indie/alternative als Top-4. Avant-Garde-Tracks sind eine massive Minderheit, die im Sample stark unterrepräsentiert ist – was die Skew in den Profil-Vektoren teilweise erklärt.

1.10.2 Was die Daten über Methodik lehren

1. **Schiefe Verteilungen brauchen Quantile-Binning.** Bei msd1m liegen 66 % aller Segmente in zwei von zehn Equal-Width-Bins. Für K-Means bedeutet das, dass 8 von 10 Features fast immer 0 sind. Quantile-Binning normalisiert das auf $\sim 10\%$ pro Bin.
2. **Skalierungs-Robustheit von Stichproben.** Die statistischen Kennwerte (Mean, Median, Std) sind zwischen msd10k und msd1m praktisch identisch. **Lessons-Learned:** schon msd10k liefert valide Schätzungen – für Trends + Verteilungen reicht es. Aber für konkrete Top-K-Listen (Aufgabe 4) ist msd1m präziser, weil die absoluten Extremwerte deutlicher sichtbar werden.

3. **Athena ist die richtige Engine für SQL-on-S3.** Auf 920 Mio Timbre-Segmenten dauern die CTAS-Queries 8-14 Sekunden. Spark hätte für die gleiche Aufgabe einen 5-minütigen Job nötig. Für reine SQL-Operationen ist Athena überlegen.



To be continued...